СОПРОВОДИТЕЛЬНАЯ ЗАПИСКА

к программному продукту

StaritsinLibrary

Специальность: 09.02.07 «Информационные системы и программирование»

Дисциплина: ПП.02.01 Технология разработки программного обеспечения

Выполнил: Старицин Олег Борисович, гр. ИПо-41

Руководитель от организации: Тюрин А.Д.

Руководитель от института: Берестов Д.Б.

Пермь, 2026

Оглавление

Оглавление 2

ВВЕДЕНИЕ 4

1.Архитектура приложения 6

2. База данных 9

2.1. Общая информация 9

2.2. Таблицы базы данных 10

2.3. Схема связей 13

2.4. Нормальные формы 14

2.5. Автоматическое создание БД 14

3. Библиотека классов 16

3.1. Модели данных (папка Models) 16

3.2. StaritsinDbContext — контекст Entity Framework Core 20

3.3. StaritsinDbContextFactory — фабрика контекста 22

3.4. PartnerRepository — репозиторий 24

3.5. DiscountService — сервис расчёта скидки 27

3.6. SaleService — сервис управления продажами 29

4. Приложение – WPF StaritsinWPF 32

4.1. App.xaml — глобальные стили 32

4.2. App.xaml.cs — точка входа 34

4.3. MainWindow — главное окно 35

4.4. PartnerFormWindow — форма партнёра 39

4.5. SalesWindow — окно управления продажами 42

4.6. SaleEditWindow — форма продажи 42

5. Модульные тесты 45
5.1. Структура тестового проекта 45
5.2. Паттерн Arrange-Act-Assert 46
5.3. Тесты расчёта цены продажи 47
Заключение 49

ВВЕДЕНИЕ

Настоящая сопроводительная записка описывает программный продукт «StaritsinPartners» — подсистему учёта партнёров компании, разработанную в рамках производственной практики ПП.02.01.

Целью разработки является создание настольного приложения, позволяющего вести учёт партнёров компании, просматривать историю реализации продукции и автоматически рассчитывать индивидуальную скидку для каждого партнёра.

Требования к системе

В соответствии с техническим заданием система должна обеспечивать следующий функционал:

№	Требование	Статус
1	Просмотр списка партнёров с карточками	Выполнено
2	Добавление данных о партнёре	Выполнено
3	Редактирование данных о партнёре	Выполнено
4	Просмотр истории реализации продукции партнёром	Выполнено
5	Расчёт индивидуальной скидки партнёра	Выполнено
6	Управление продажами (добавление/редактирование/удаление)	Выполнено
7	База данных в 3НФ с ссылочной целостностью	Выполнено
8	Именование по стандарту CamelCase	Выполнено
9	Модульные тесты	Выполнено

10	Обработка исключений и валидация ввода	Выполнено

Использованные технологии

Компонент	Технология	Версия
Язык программирования	C#	.NET 8
UI-фреймворк	WPF (Windows Presentation Foundation)	.NET 8
ORM	Entity Framework Core	6.0.0
СУБД	PostgreSQL	15+
Провайдер PostgreSQL	Npgsql	6.0.0
Фреймворк тестирования	MSTest	3.1.1
Среда разработки	Visual Studio 2022	Community
Контроль версий	Git / GitHub	-

1.Архитектура приложения

Приложение построено по принципу многослойной архитектуры (Layered Architecture) и состоит из трёх отдельных проектов (сборок), объединённых в одно решение Visual Studio.

Ключевой принцип: WPF-проект не содержит прямых зависимостей от Entity Framework Core. Вся работа с базой данных инкапсулирована в библиотеке StaritsinPartners.Domain.

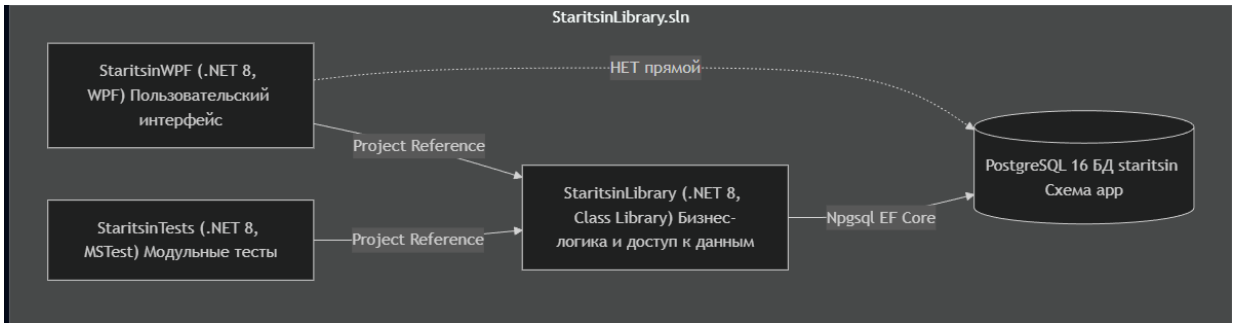

Диаграмма 1 — Структура решения (Component Diagram)

Такое разделение обеспечивает независимость бизнес-логики от интерфейса. Библиотека Domain может быть использована с любым другим UI-фреймворком (например, MAUI, WinForms, ASP.NET) без изменений.

Состав проектов

Проект	Тип	Назначение
StaritsinPartners.Domain	Class Library (.NET 8)	Модели данных, контекст EF Core, фабрика, репозиторий, сервисы скидок и продаж, DTO-объекты
StaritsinPartners.WPF	WPF Application (.NET 8)	Все окна и формы приложения. Не

		содержит ссылок на EF Core или Npgsql
StaritsinPartners.Tests	MSTest Project (.NET 8)	72 модульных теста. Тестирует только Domain без запуска UI и без обращений к БД

Зависимости NuGet

Проект	Пакет	Версия	Назначение
Domain	Microsoft.EntityFrameworkCore	6.0.0	ORM для работы с БД
Domain	Npgsql.EntityFrameworkCore.PostgreSQL	6.0.0	Провайдер PostgreSQL
WPF	Нет зависимостей EF	—	Только ссылка на Domain
Tests	MSTest.TestFramework	3.1.1	Фреймворк тестирования
Tests	MSTest.TestAdapter	3.1.1	Адаптер для Test Explorer
Tests	Microsoft.NET.Test.Sdk	17.8.0	SDK тестирования

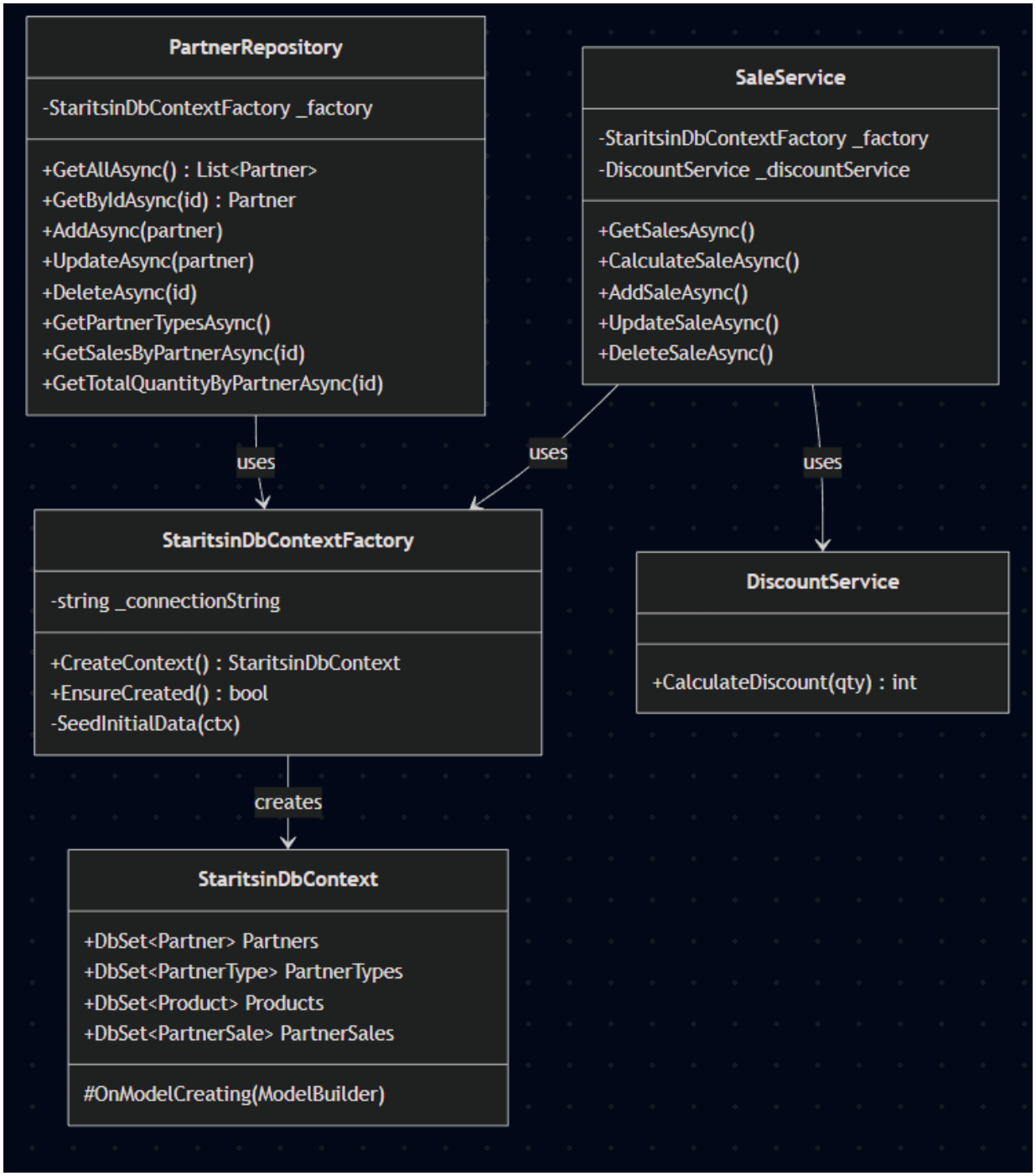

Диаграмма 2 — Диаграмма классов (обзор всех слоёв)

2. База данных

2.1. Общая информация

Параметр	Значение
СУБД	PostgreSQL 15
База данных	staritsin
Схема	app
Роль	app (пароль: 123456789)
Нормальная форма	3НФ (третья нормальная форма)
Ссылочная целостность	Обеспечена через внешние ключи

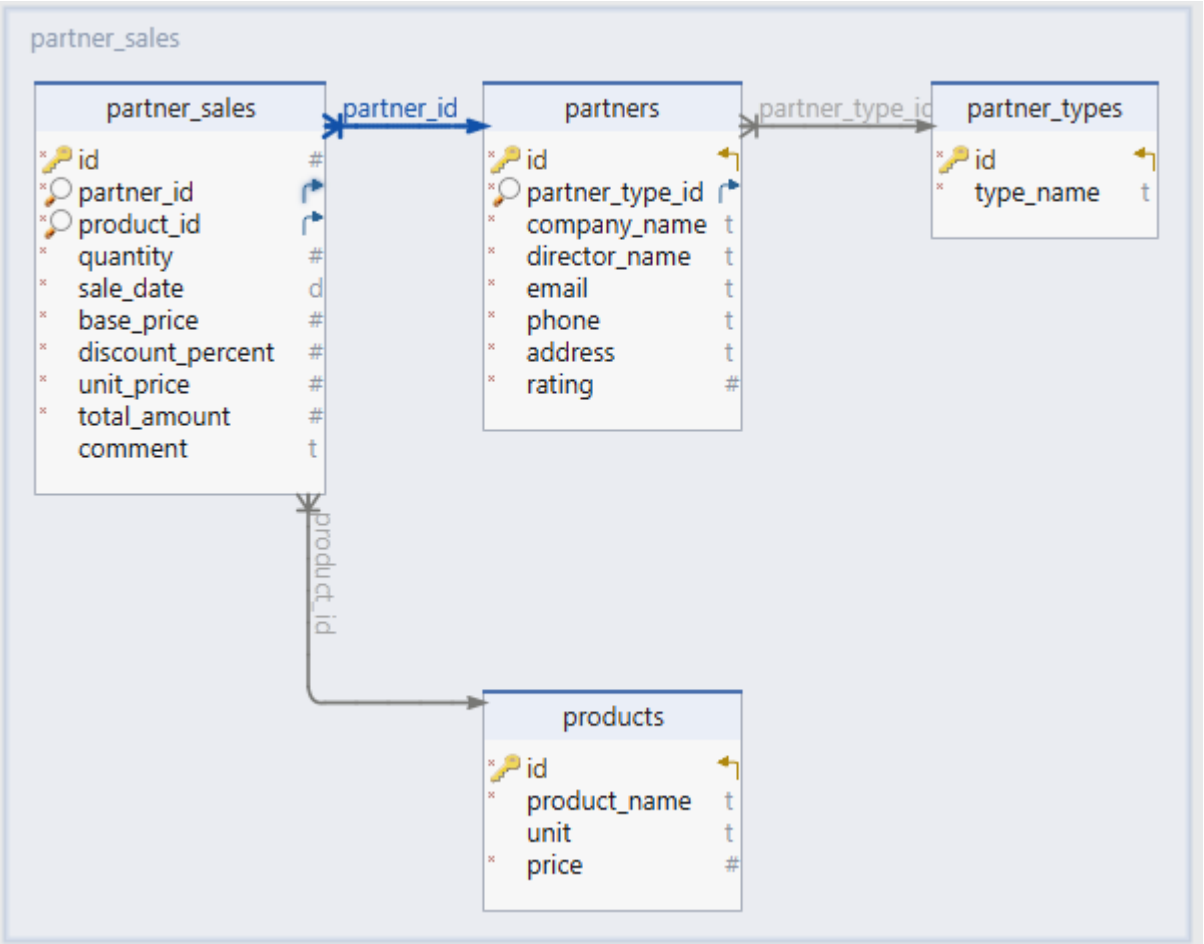

Рисунок 1 — ER-диаграмма базы данных staritsin

2.2. Таблицы базы данных

Таблица app.partner_types

Справочник типов партнёров. Значения типов (ООО, ЗАО, ПАО, ОАО) вынесены в отдельную таблицу в соответствии с требованием 3НФ — исключение повторяющихся групп данных.

Колонка	Тип	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Первичный ключ, автоинкремент
type_name	VARCHAR(100)	NOT NULL, UNIQUE	Наименование типа партнёра

Таблица app.partners

Основная таблица партнёров. Хранит все данные о компании-партнёре. Поле partner_type_id — внешний ключ на таблицу partner_types. Ограничение CHECK на поле rating гарантирует неотрицательность рейтинга на уровне БД.

Колонка	Тип	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Первичный ключ
partner_type_id	INTEGER	NOT NULL, FK → partner_types.id	Тип партнёра
company_name	VARCHAR(255)	NOT NULL	Наименование компании
director_name	VARCHAR(255)	NOT NULL	ФИО директора
email	VARCHAR(255)	NOT NULL	Электронная почта

phone	VARCHAR(50)	NOT NULL	Телефон
address	VARCHAR(500)	NOT NULL	Юридический адрес
rating	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0	Рейтинг партнёра

Таблица app.products

Каталог продуктов компании. Поле price хранит базовую цену продукта. При расчёте стоимости продажи цена берётся из этой таблицы и сохраняется в запись о продаже — это обеспечивает хранение исторической цены.

id	SERIAL	PRIMARY KEY	Первичный ключ
product_name	VARCHAR(255)	NOT NULL	Наименование продукта
unit	VARCHAR(50)	NOT NULL, DEFAULT «шт»	Единица измерения
price	NUMERIC(12,2)	NOT NULL, DEFAULT 0, CHECK ≥ 0	Базовая цена

Таблица app.partner_sales

Центральная таблица фактов. Хранит записи о каждой продаже. Ключевая особенность: поля base_price, discount_percent и unit_price

записываются в момент продажи и не пересчитываются при изменении цены продукта или объёма продаж партнёра. Это обеспечивает историческую достоверность данных.

Колонка	Тип	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Первичный ключ
partner_id	INTEGER	NOT NULL, FK CASCADE	Партнёр (каскадное удаление)
product_id	INTEGER	NOT NULL, FK	Продукт
quantity	INTEGER	NOT NULL, CHECK > 0	Количество
sale_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Дата продажи
base_price	NUMERIC(12,2)	NOT NULL, DEFAULT 0	Базовая цена на момент продажи
discount_percent	INTEGER	NOT NULL, DEFAULT 0, CHECK 0–100	Скидка на момент продажи

unit_price	NUMERIC(12,2)	NOT NULL, DEFAULT 0	Цена с учётом скидки
total_amount	NUMERIC(14,2)	NOT NULL, DEFAULT 0	Итоговая сумма
comment	TEXT	NULL	Комментарий к продаже

2.3. Схема связей

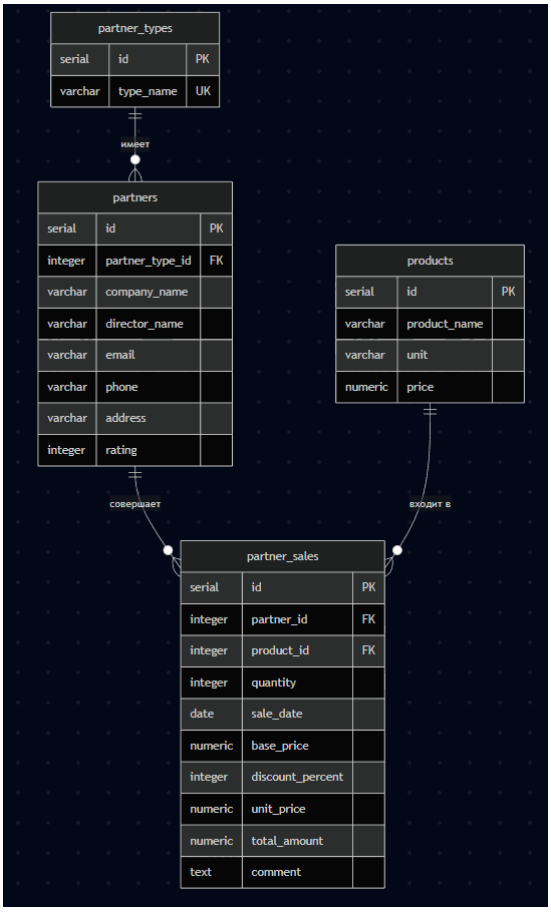

Диаграмма 3 — Схема связей между таблицами (ERD)

2.4. Нормальные формы

База данных приведена к третьей нормальной форме (3НФ). Проверка:

1НФ — все атрибуты атомарны (нет составных или многозначных полей), каждая таблица имеет первичный ключ.

2НФ — все неключевые атрибуты полностью функционально зависят от первичного ключа (все ключи простые — SERIAL, поэтому 2НФ выполняется автоматически).

3НФ — нет транзитивных зависимостей. Тип партнёра вынесен в отдельную таблицу partner_types: название типа зависит от type_id, а не от id партнёра. Все остальные атрибуты таблиц зависят только от первичного ключа.

2.5. Автоматическое создание БД

При первом запуске приложение автоматически создаёт схему app и все таблицы через метод StaritsinDbContextFactory.EnsureCreated(). Это избавляет пользователя от необходимости выполнять скрипты вручную. Порядок действий при первом запуске:

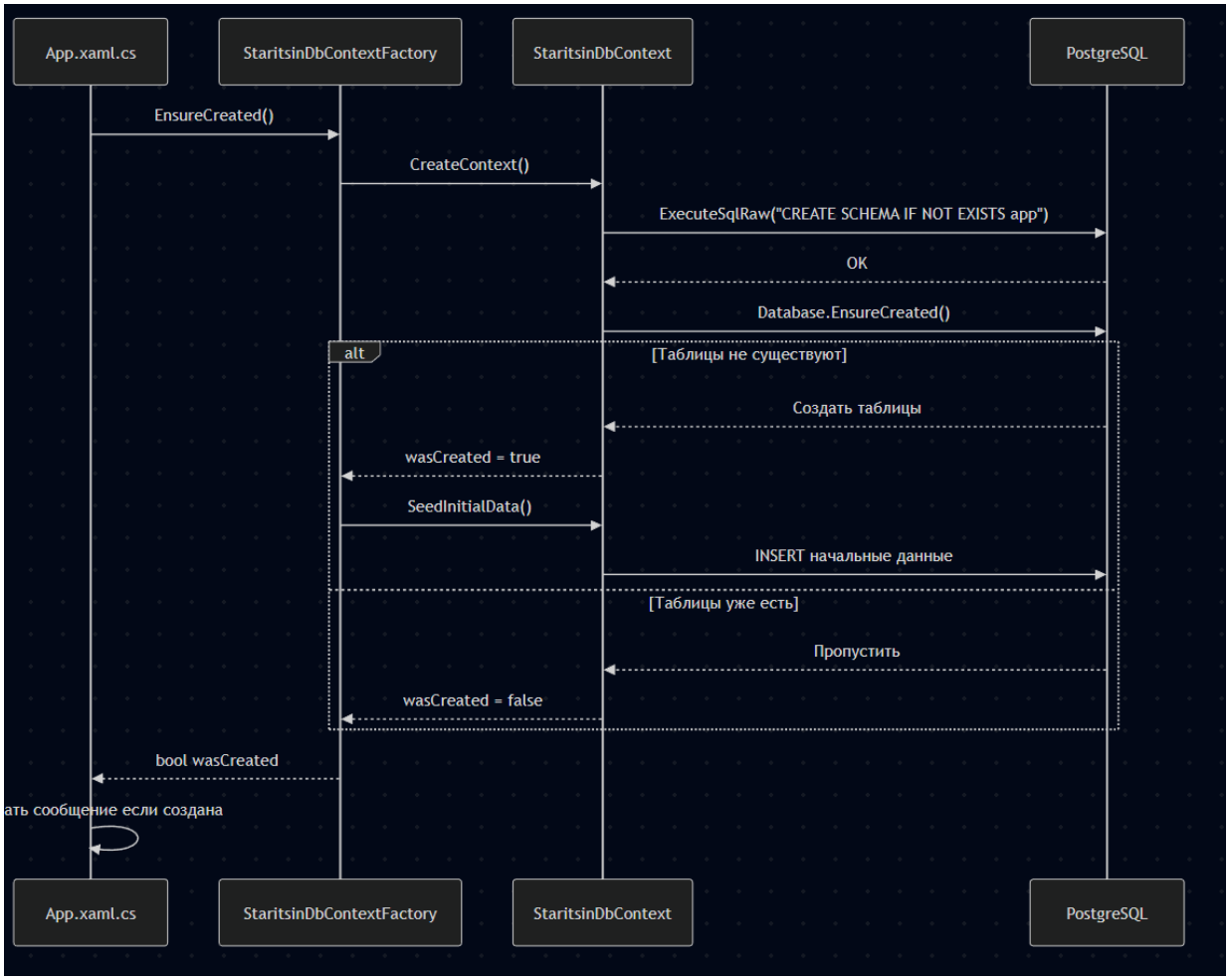

Диаграмма 4 — Последовательность инициализации БД при первом запуске

3. Библиотека классов

Библиотека классов является центральным компонентом системы. Она содержит всю бизнес-логику и полностью изолирует WPF-приложение от деталей работы с базой данных.

3.1. Модели данных (папка Models)

Модели данных — это классы C#, которые Entity Framework Core отображает на таблицы PostgreSQL. Каждый класс соответствует одной таблице в схеме app.

Класс Partner

Соответствует таблице app.partners. Содержит атрибуты валидации из пространства имён System.ComponentModel.DataAnnotations:

Partner.cs — ключевые поля и валидация

```
public class Partner
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Укажите тип партнёра")]
    public int PartnerTypeId { get; set; }

    [Required(ErrorMessage = "Укажите наименование")]
    [MaxLength(255)]
    public string CompanyName { get; set; } = string.Empty;

    [Required, MaxLength(255)]
    public string DirectorName { get; set; } = string.Empty;

    [Required, MaxLength(255)]
    public string Email { get; set; } = string.Empty;

    [Required, MaxLength(50)]
```

```
    public string Phone { get; set; } = string.Empty;

    [Required, MaxLength(500)]
    public string Address { get; set; } = string.Empty;
    [Range(0, int.MaxValue, ErrorMessage = "Рейтинг >= 0")]
    public int Rating { get; set; }
    public PartnerType PartnerType { get; set; }
    public ICollection<PartnerSale> Sales { get; set; }
        = new List<PartnerSale>();
}
```

Класс PartnerSale

Соответствует таблице app.partner_sales. Содержит поля для хранения исторических цен. Особенность: BasePrice, DiscountPercent и UnitPrice сохраняются в момент оформления продажи и не изменяются при последующем изменении цены продукта.

PartnerSale.cs — поля исторических цен

```
public class PartnerSale
{
    public int Id { get; set; }
    public int PartnerId { get; set; }
    public int ProductId { get; set; }
    public int Quantity { get; set; }
    public DateTime SaleDate { get; set; } = DateTime.Today;
    public decimal BasePrice { get; set; }      // цена до скидки
    public int DiscountPercent { get; set; }    // скидка партнёра
    public decimal UnitPrice { get; set; }      // цена со скидкой
    public decimal TotalAmount { get; set; }    // Quantity * UnitPrice
    public string Comment { get; set; }
```

```
    public Partner Partner { get; set; }
    public Product Product { get; set; }
}
```

DTO-объекты

Помимо сущностей-моделей, в папке Models находятся DTO (Data Transfer Object) — вспомогательные классы для передачи данных между слоями:

Класс	Назначение
SaleEditModel	Данные формы при добавлении/редактировании продажи
SaleListItemDTO	Строка в таблице всех продаж (включает имена партнёра и продукта)
SaleCalculationResultDTO	Результат расчёта: BasePrice, DiscountPercent, UnitPrice, TotalAmount
SalePartnerLookupDTO	Элемент ComboBox партнёров в форме продажи: Id + CompanyName
SaleProductLookupDTO	Элемент ComboBox продуктов в форме продажи: Id + ProductName + Price

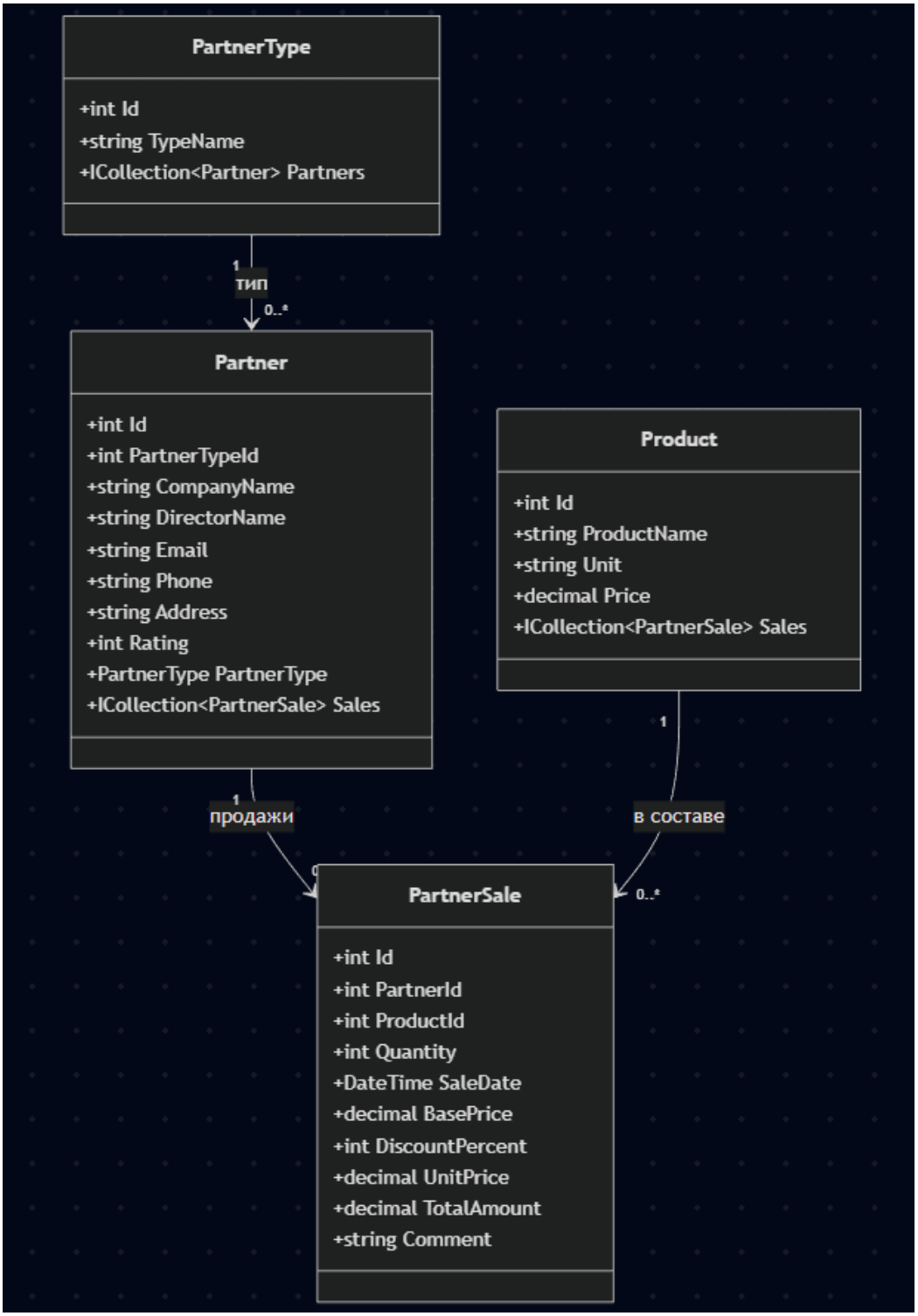

Диаграмма 5 — Диаграмма классов моделей данных

3.2. StaritsinDbContext — контекст Entity Framework Core

Класс StaritsinDbContext является центральной точкой взаимодействия с базой данных. Он наследует DbContext из Entity Framework Core и настраивает маппинг C#-моделей на таблицы PostgreSQL.

Свойства DbSet

Каждое свойство DbSet<T> представляет собой «виртуальную таблицу» — через него EF Core строит и выполняет SQL-запросы:

Свойства DbSet в StaritsinDbContext

```
public DbSet<Partner>     Partners     { get; set; }
public DbSet<PartnerType> PartnerTypes { get; set; }
public DbSet<Product>     Products     { get; set; }
public DbSet<PartnerSale> PartnerSales { get; set; }
```

Настройка маппинга (OnModelCreating)

Метод OnModelCreating вызывается EF Core при инициализации контекста. В нём настраиваются все детали отображения классов на таблицы PostgreSQL:

Маппинг таблицы partners

```
modelBuilder.HasDefaultSchema("app");  // схема app для всех таблиц

modelBuilder.Entity<Partner>(entity =>
{
    entity.ToTable("partners");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Id).HasColumnName("id");
    entity.Property(e => e.CompanyName)
        .HasColumnName("company_name")
        .HasMaxLength(255).IsRequired();
    entity.Property(e => e.Rating)
        .HasColumnName("rating").HasDefaultValue(0);
```

```
    entity.HasOne(e => e.PartnerType)
        .WithMany(t => t.Partners)
        .HasForeignKey(e => e.PartnerTypeId);
});
```

Маппинг таблицы partner_sales (с денежными полями)

```
modelBuilder.Entity<PartnerSale>(entity =>
{
    entity.ToTable("partner_sales");
    entity.Property(e => e.BasePrice)
        .HasColumnName("base_price")
        .HasColumnType("numeric(12,2)").HasDefaultValue(0m);
    entity.Property(e => e.TotalAmount)
        .HasColumnName("total_amount")
        .HasColumnType("numeric(14,2)").HasDefaultValue(0m);
    entity.HasOne(e => e.Partner)
        .WithMany(p => p.Sales)
        .HasForeignKey(e => e.PartnerId)
        .OnDelete(DeleteBehavior.Cascade);
});
```

3.3. StaritsinDbContextFactory — фабрика контекста

Класс StaritsinDbContextFactory реализует паттерн «Фабрика» (Factory Pattern) и решает ключевую архитектурную задачу: WPF-проект не должен содержать прямых зависимостей от Entity Framework Core.

Паттерн Factory

Фабрика принимает строку подключения в конструкторе и предоставляет метод CreateContext() для создания новых экземпляров StaritsinDbContext. Репозиторий и сервисы используют фабрику вместо прямого создания контекста:

Использование фабрики в репозитории

```
public async Task<List<Partner>> GetAllAsync()
{
    using (var context = _factory.CreateContext())
    {
        return await context.Partners
            .Include(p => p.PartnerType)
            .OrderBy(p => p.CompanyName)
            .ToListAsync();
    }
}
```

Метод EnsureCreated

Метод EnsureCreated() выполняет три последовательных действия при первом запуске:

Метод EnsureCreated() — автоматическое создание БД

```
public bool EnsureCreated()
{
```

```
    using (var context = CreateContext())
    {
        context.Database.ExecuteSqlRaw(
            "CREATE SCHEMA IF NOT EXISTS app");
        bool wasCreated = context.Database.EnsureCreated();
        if (wasCreated)
            SeedInitialData(context);

        return wasCreated;
    }
}
```

Метод SeedInitialData

При первом создании БД автоматически добавляются тестовые данные: 4 типа партнёров (ЗАО, ООО, ПАО, ОАО), 4 продукта, 3 тестовых партнёра и 5 тестовых продаж с разным объёмом (от 3000 до 62000 единиц), что позволяет сразу видеть разные уровни скидок.

3.4. PartnerRepository — репозиторий

Репозиторий реализует паттерн «Репозиторий» (Repository Pattern) и инкапсулирует все CRUD-операции над партнёрами. WPF-приложение вызывает только методы репозитория и ничего не знает о SQL-запросах.

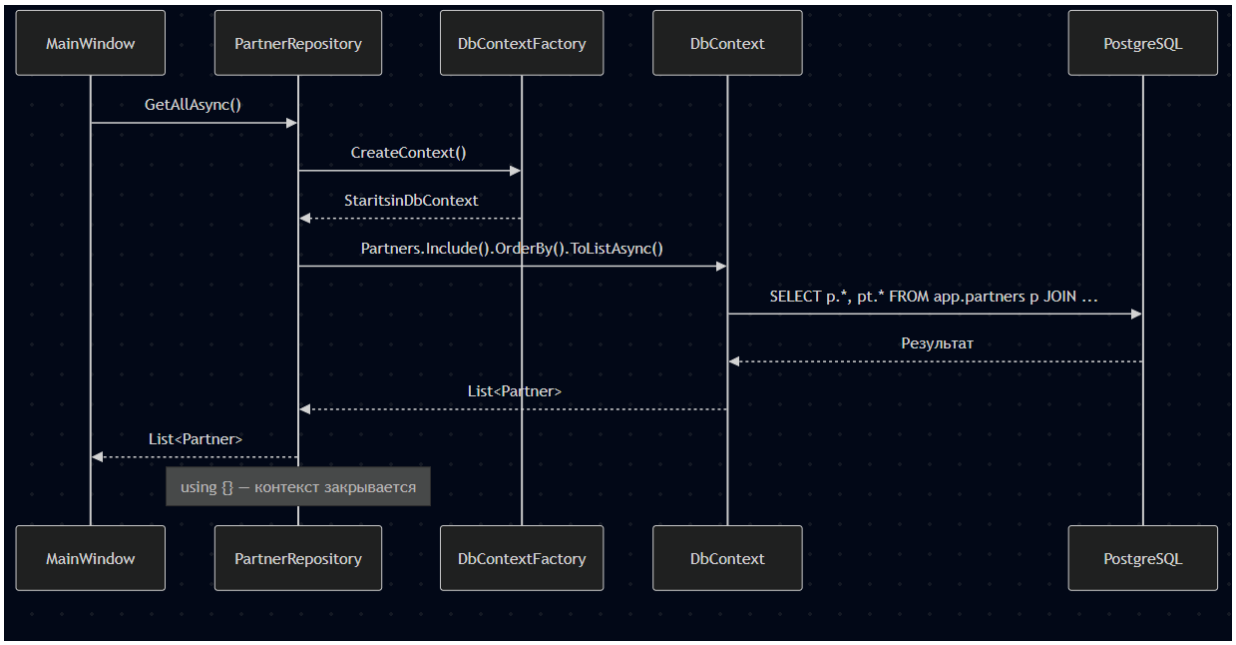

Диаграмма 6 — Взаимодействие репозитория с остальными компонентами

Описание методов репозитория

Метод	SQL-эквивалент	Описание
GetAllAsync()	SELECT * JOIN partner_types ORDER BY company_name	Все партнёры с типами. Include()

		создаёт JOIN
GetByIdAsync(id)	SELECT * WHERE id=? + JOIN + JOIN	Один партнёр с типом и всеми продажами через ThenInclude
AddAsync(partner)	INSERT INTO partners ...	EF Core генерирует INSERT, возвращает Id
UpdateAsync(partner)	UPDATE partners SET ... WHERE id=?	EF Core отслеживает изменённые поля
DeleteAsync(id)	DELETE FROM partners WHERE id=? (CASCADE)	Каскадно удаляет все продажи партнёра
GetPartnerTypesAsync()	SELECT * FROM partner_types ORDER BY type_name	Для ComboBox в форме партнёра

GetSalesByPartnerAsync(id)	SELECT * FROM partner_sales WHERE partner_id=? JOIN products ORDER BY sale_date DESC	История продаж партнёра
GetTotalQuantityByPartnerAsync(id)	SELECT SUM(quantity) FROM partner_sales WHERE partner_id=?	SUM для расчёта скидки. ?? 0 защита от NULL

GetTotalQuantityByPartnerAsync — защита от NULL

```
public async Task<int> GetTotalQuantityByPartnerAsync(int partnerId)
{
    using (var context = _factory.CreateContext())
    {
        var sum = await context.PartnerSales
            .Where(s => s.PartnerId == partnerId)
            .SumAsync(s => (int?)s.Quantity);

        return sum ?? 0;
    }
}
```

3.5. DiscountService — сервис расчёта скидки

Простой сервис с единственным публичным методом CalculateDiscount(). Реализует бизнес-алгоритм расчёта индивидуальной скидки партнёра на основе суммарного объёма реализованной продукции.

Суммарное количество	Скидка	Пример
0 — 9 999	0%	Партнёр продал 3 000 ед. → скидки нет
10 000 — 49 999	5%	Партнёр продал 23 000 ед. → скидка 5%
50 000 — 299 999	10%	Партнёр продал 62 000 ед. → скидка 10%
300 000 и более	15%	Партнёр продал 350 000 ед. → скидка 15%

DiscountService.CalculateDiscount() — реализация алгоритма

```
public int CalculateDiscount(int totalQuantity)
{
    if (totalQuantity < 10000)   return 0;
    if (totalQuantity < 50000)   return 5;
    if (totalQuantity < 300000)  return 10;
    return 15;  // >= 300 000
}
```

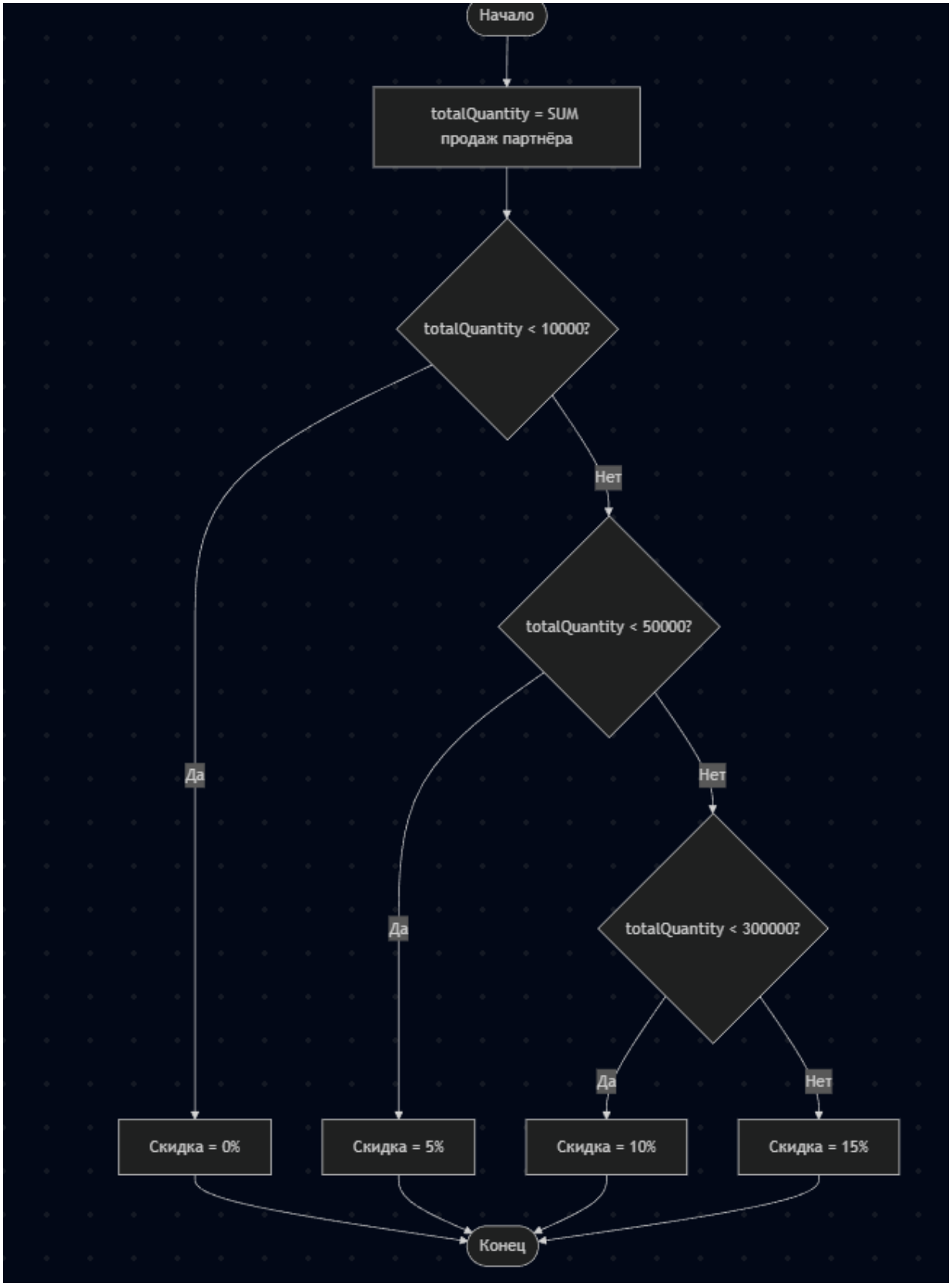

Диаграмма 7 — Блок-схема алгоритма расчёта скидки

3.6. SaleService — сервис управления продажами

SaleService — наиболее сложный класс в Domain. Он принимает StaritsinDbContextFactory и DiscountService и реализует полный цикл работы с продажами, включая расчёт цены с учётом скидки.

Метод CalculateSaleAsync — расчёт стоимости продажи

Ключевой метод сервиса. Алгоритм работы:

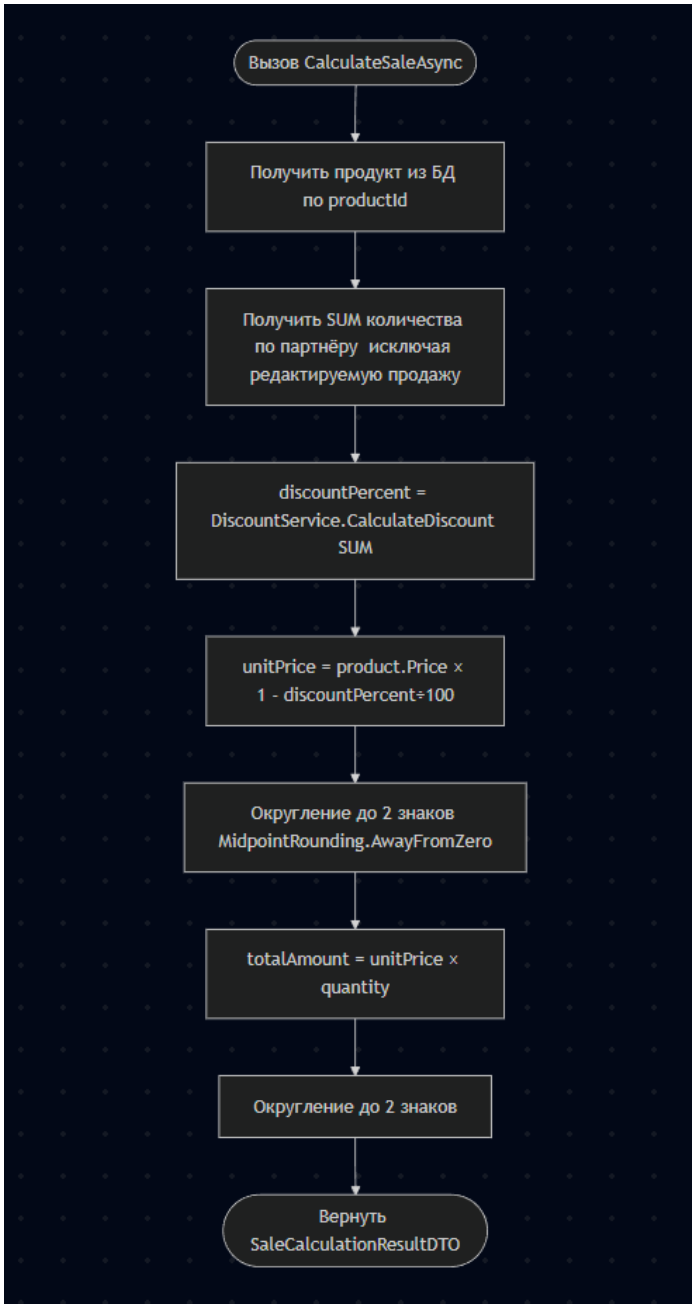

Диаграмма 8 — Алгоритм расчёта стоимости продажи

CalculateSaleAsync — формула расчёта цены

```
var totalQty = await context.PartnerSales
    .Where(x => x.PartnerId == partnerId &&
            (!editingSaleId.HasValue || x.Id != editingSaleId.Value))
    .SumAsync(x => (int?)x.Quantity) ?? 0;

int discountPercent = _discountService.CalculateDiscount(totalQty);
decimal unitPrice = Math.Round(
    product.Price * (1m - discountPercent / 100m), 2,
    MidpointRounding.AwayFromZero);

decimal totalAmount = Math.Round(
    unitPrice * quantity, 2,
    MidpointRounding.AwayFromZero);
```

Метод AddSaleAsync — добавление продажи

При добавлении продажи рассчитанные значения сохраняются вместе с записью. Именно в этот момент фиксируется историческая цена — даже если завтра цена продукта изменится, запись о прошлой продаже останется неизменной:

AddSaleAsync — фиксация исторической цены

```
var calculation = await CalculateSaleAsync(
    model.PartnerId, model.ProductId, model.Quantity);

var entity = new PartnerSale
{
    PartnerId       = model.PartnerId,
    ProductId       = model.ProductId,
    Quantity        = model.Quantity,
    SaleDate        = model.SaleDate,
```

```
    BasePrice       = calculation.BasePrice,
    DiscountPercent = calculation.DiscountPercent,
    UnitPrice       = calculation.UnitPrice,
    TotalAmount     = calculation.TotalAmount,
    Comment         = NormalizeNullable(model.Comment)
};
context.PartnerSales.Add(entity);
await context.SaveChangesAsync();
```

4. Приложение – WPF StaritsinWPF

WPF-приложение отвечает исключительно за пользовательский интерфейс. Все зависимости от Entity Framework Core, Npgsql и базы данных отсутствуют — приложение работает только через публичные методы классов из StaritsinPartners.Domain.

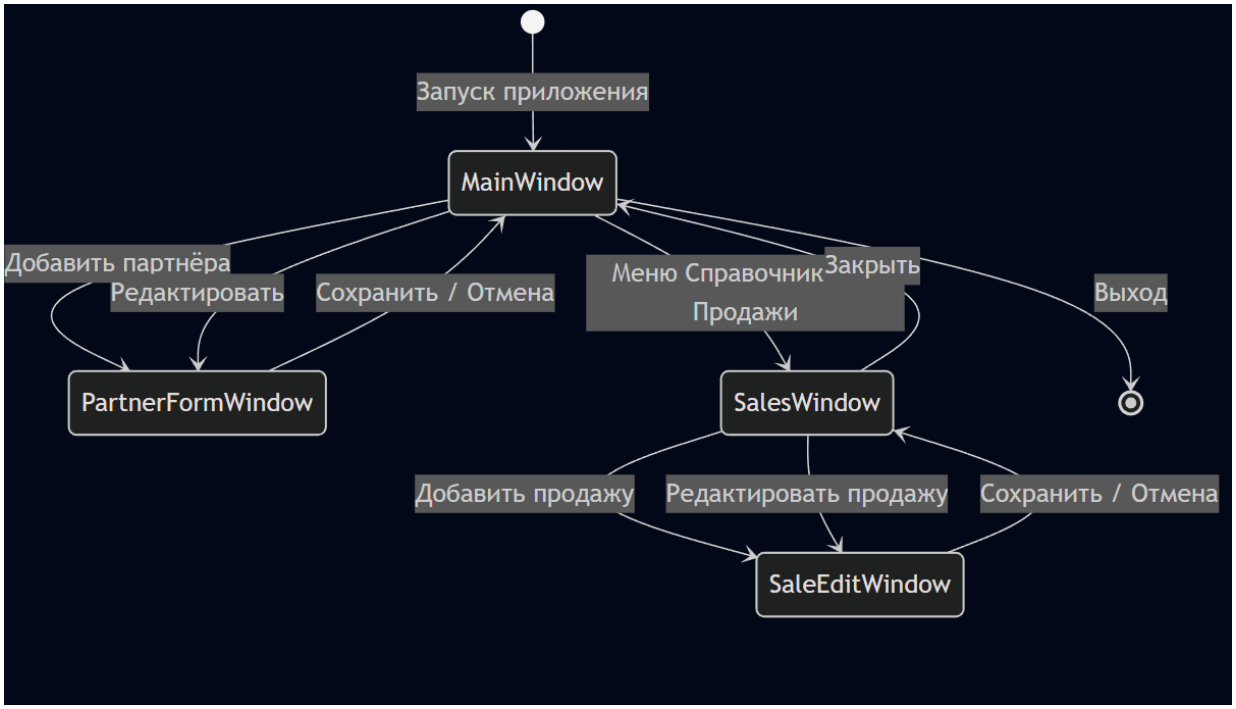

Диаграмма 9 — Навигация между окнами приложения

4.1. App.xaml — глобальные стили

App.xaml содержит ResourceDictionary с глобальными стилями, которые автоматически применяются ко всем элементам управления во всех окнах приложения. Это обеспечивает единый визуальный стиль.

Цветовая палитра приложения:

Ключ ресурса	Цвет	Применение
PrimaryBrush	#67BA80 (зелёный)	Фон заголовков окон, основная кнопка
PrimaryDarkBrush	#4A9E63 (тёмно-зелёный)	Hover-состояние кнопки, текст скидки

PrimaryLightBrush	#F0FAF3 (светло-зелёный)	Фон при наведении на карточку
BackgroundBrush	#F5F5F5 (светло-серый)	Фон всех окон
CardBrush	#FFFFFF (белый)	Фон карточек партнёров
TextPrimaryBrush	#2B2B2B (почти чёрный)	Основной текст
TextSecondaryBrush	#777777 (серый)	Вспомогательный текст

Стили переопределены для TextBox (рамка подсвечивается зелёным при фокусе), ComboBox, DataGrid (чередование строк, зелёные заголовки столбцов), кнопок (PrimaryButtonStyle и SecondaryButtonStyle).

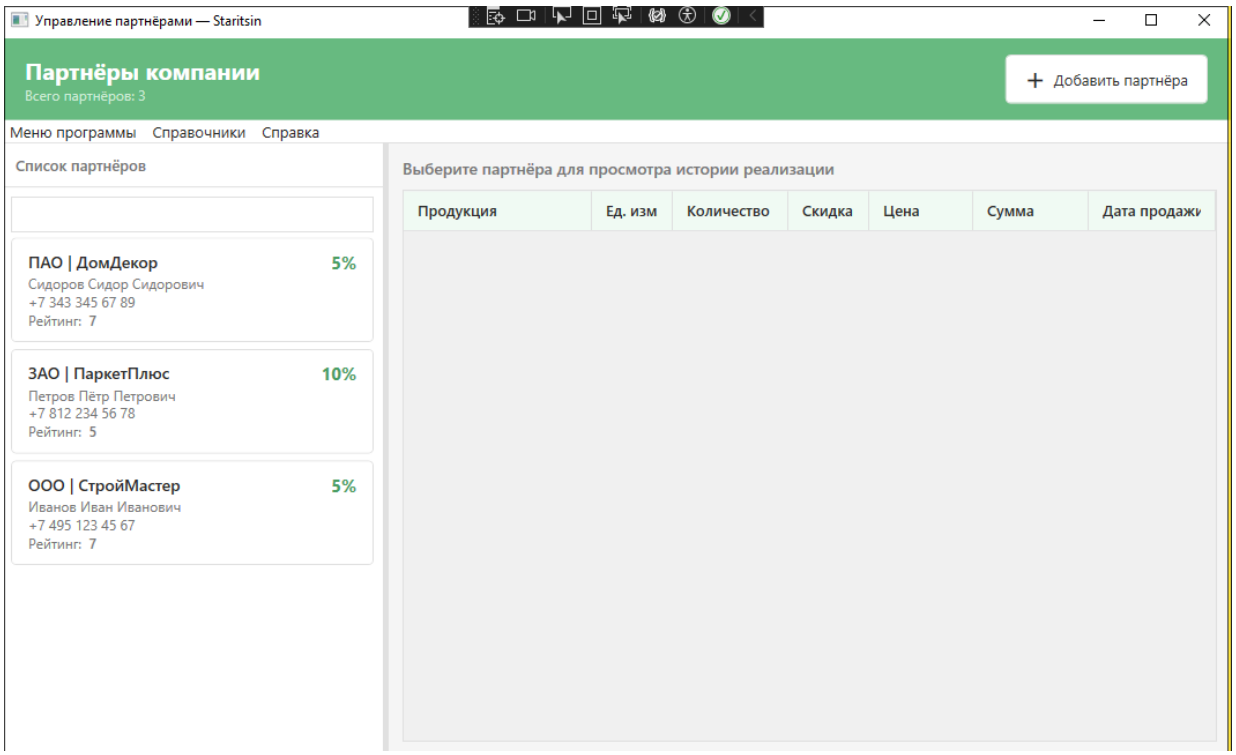

Рисунок 3 — Главное окно приложения MainWindow с карточками партнёров

4.2. App.xaml.cs — точка входа

Метод OnStartup() выполняется при запуске приложения. Здесь создаются все зависимости и проверяется доступность базы данных:

App.xaml.cs — инициализация сервисов

```
protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);
    const string connectionString =

"Host=localhost;Port=5432;Database=staritsin;Username=app;Password=123456789;";
    var factory = new StaritsinDbContextFactory(connectionString);

    // Создаём объекты сервисов — доступны через App.XXX из любого окна
    PartnerRepository = new PartnerRepository(factory);
    DiscountService   = new DiscountService();
    SaleService       = new SaleService(factory, DiscountService);
    bool wasCreated = factory.EnsureCreated();
    if (wasCreated)
        MessageBox.Show("База данных создана автоматически");
}
```

Три статических свойства App.PartnerRepository, App.DiscountService и App.SaleService доступны из любого окна приложения через App.Current или напрямую через имя класса. Это упрощённый способ разделения зависимостей без использования DI-контейнера.

4.3. MainWindow — главное окно

Главное окно является точкой входа пользователя в систему. Состоит из двух панелей, разделённых GridSplitter.

Карточка партнёра (DataTemplate)

Каждый партнёр отображается в виде карточки. DataTemplate описывает внешний вид одного элемента списка:

Элемент карточки	Привязка (Binding)	Описание
Заголовок жирный	{Binding DisplayTitle}	"ООО | СтройМастер" — из PartnerViewModel
ФИО директора	{Binding DirectorName}	Навигационное свойство через PartnerViewModel
Телефон	{Binding Phone}	
Рейтинг	{Binding Rating}	Целое число
Скидка (правый угол)	{Binding DiscountText}	"10%" зелёным цветом, жирный

Класс PartnerViewModel

PartnerViewModel — вспомогательный класс, объявленный в файле MainWindow.xaml.cs. Он оборачивает модель Partner и добавляет вычисляемые свойства, необходимые для отображения в XAML:

PartnerViewModel — вычисляемые свойства для XAML

```
public class PartnerViewModel
{
    private readonly Partner _partner;
    public PartnerViewModel(Partner partner, int discount)
    { _partner = partner; Discount = discount; }
```

```
    // Базовые свойства — проксирование из модели
    public int    Id          => _partner.Id;
    public string CompanyName => _partner.CompanyName;
    public string Phone       => _partner.Phone;
    public int    Rating      => _partner.Rating;
    public string DisplayTitle =>
        _partner.PartnerType?.TypeName + " | " + _partner.CompanyName;

    public string DiscountText => Discount + "%";
    public int    Discount { get; }
}
```

Метод LoadPartnersAsync

Основной метод загрузки данных. Для каждого партнёра асинхронно запрашивается суммарное количество продаж, после чего вычисляется скидка:

LoadPartnersAsync — загрузка и расчёт скидок

```
private async Task LoadPartnersAsync()
{
    var partners = await _partnerRepository.GetAllAsync();
    _allPartners = new List<PartnerViewModel>();

    foreach (var partner in partners)
    {
        // Отдельный запрос к БД для каждого партнёра
        int totalQty = await _partnerRepository
                        .GetTotalQuantityByPartnerAsync(partner.Id);
        int discount  = _discountService.CalculateDiscount(totalQty);
        _allPartners.Add(new PartnerViewModel(partner, discount));
    }

    ApplySearchFilter(SearchBox.Text);  // локальная фильтрация
```

```
    PartnerCountText.Text = "Всего: " + _allPartners.Count;
}
```

Поиск по наименованию

Метод ApplySearchFilter() выполняет фильтрацию локально (без запроса к БД) по уже загруженному списку _allPartners. Используется метод IndexOf с флагом OrdinalIgnoreCase для регистронезависимого сравнения:

ApplySearchFilter — локальная фильтрация без запроса к БД

```
private void ApplySearchFilter(string searchText)
{
    List<PartnerViewModel> result;
    if (string.IsNullOrWhiteSpace(searchText))
        result = _allPartners;
    else
        result = _allPartners
            .Where(p => p.CompanyName.IndexOf(searchText,
                StringComparison.OrdinalIgnoreCase) >= 0)
            .ToList();
    PartnersItemsControl.ItemsSource = result;
}
```

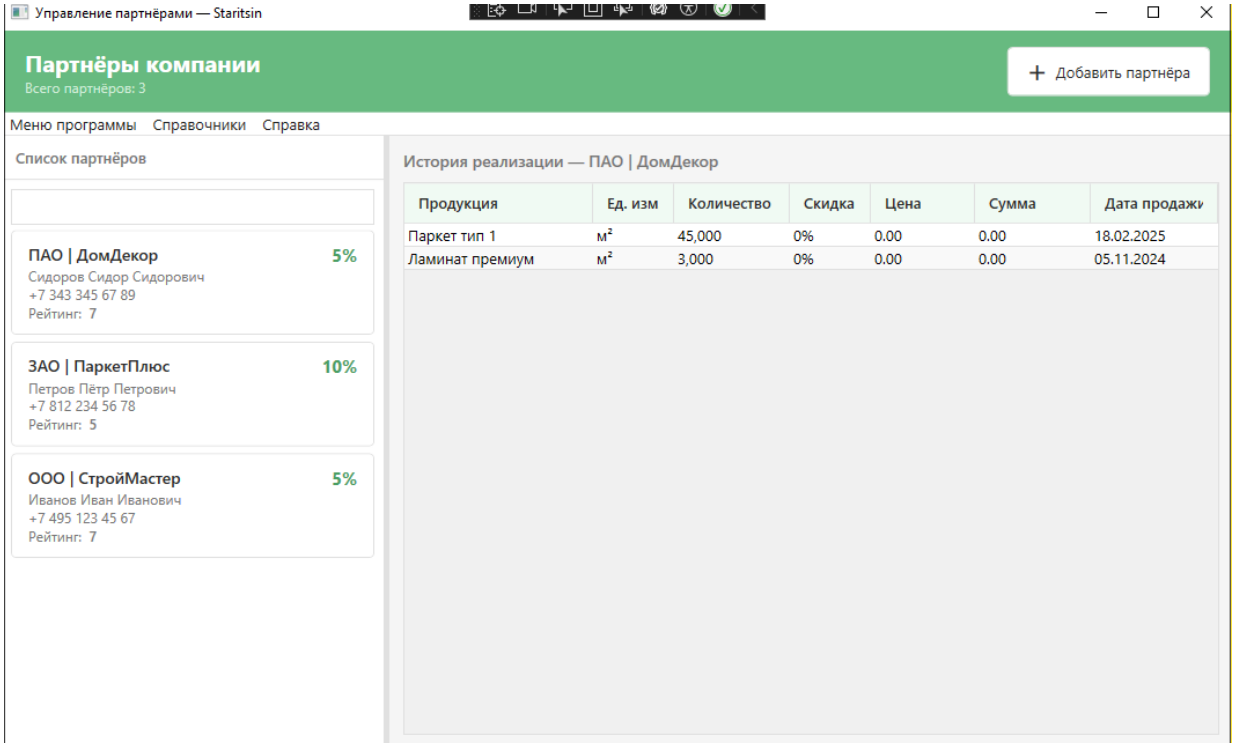

Рисунок 4 — Главное окно с выбранным партнёром и историей продаж справа

4.4. PartnerFormWindow — форма партнёра

Форма добавления и редактирования партнёра. Работает в двух режимах: добавление (editPartnerId = null) и редактирование (editPartnerId = int).

Поля формы

Поле	Элемент XAML	Валидация
Наименование компании *	TextBox	Обязательное
Тип партнёра *	ComboBox (из БД)	Выбор из списка обязателен
ФИО директора *	TextBox	Обязательное
Телефон *	TextBox + ToolTip	Обязательное
Email *	TextBox + ToolTip	Обязательное + формат через Regex
Юридический адрес *	TextBox (многострочный)	Обязательное
Рейтинг *	TextBox + PreviewTextInput	Только цифры, целое ≥ 0

Валидация в ValidateForm()

Метод ValidateForm() проверяет все поля и формирует единое сводное сообщение об ошибках. Пользователь видит список всех нарушений сразу, а не по одному:

ValidateForm — сводная валидация всех полей

```
private bool ValidateForm(out string errorMessage)
{
    var errors = new StringBuilder();
```

```
        if (string.IsNullOrWhiteSpace(CompanyNameBox.Text))
            errors.AppendLine("• Укажите наименование компании.");

        if (PartnerTypeCombo.SelectedItem == null)
            errors.AppendLine("• Выберите тип партнёра из списка.");

        // Валидация email через регулярное выражение
        if (!IsValidEmail(EmailBox.Text.Trim()))
            errors.AppendLine("• Email имеет неверный формат.");

        // Валидация рейтинга
        int rating;
        if (!int.TryParse(RatingBox.Text.Trim(), out rating) || rating < 0)
            errors.AppendLine("• Рейтинг: целое число >= 0.");

        if (errors.Length > 0) {
            errorMessage = "Исправьте ошибки:\n\n" + errors;
            return false;
        }
        errorMessage = string.Empty;
        return true;
    }
```

Ограничение ввода в поле рейтинга реализовано через обработчик PreviewTextInput — символы не из разряда цифр блокируются ещё до попадания в TextBox, что является визуальной подсказкой для пользователя.

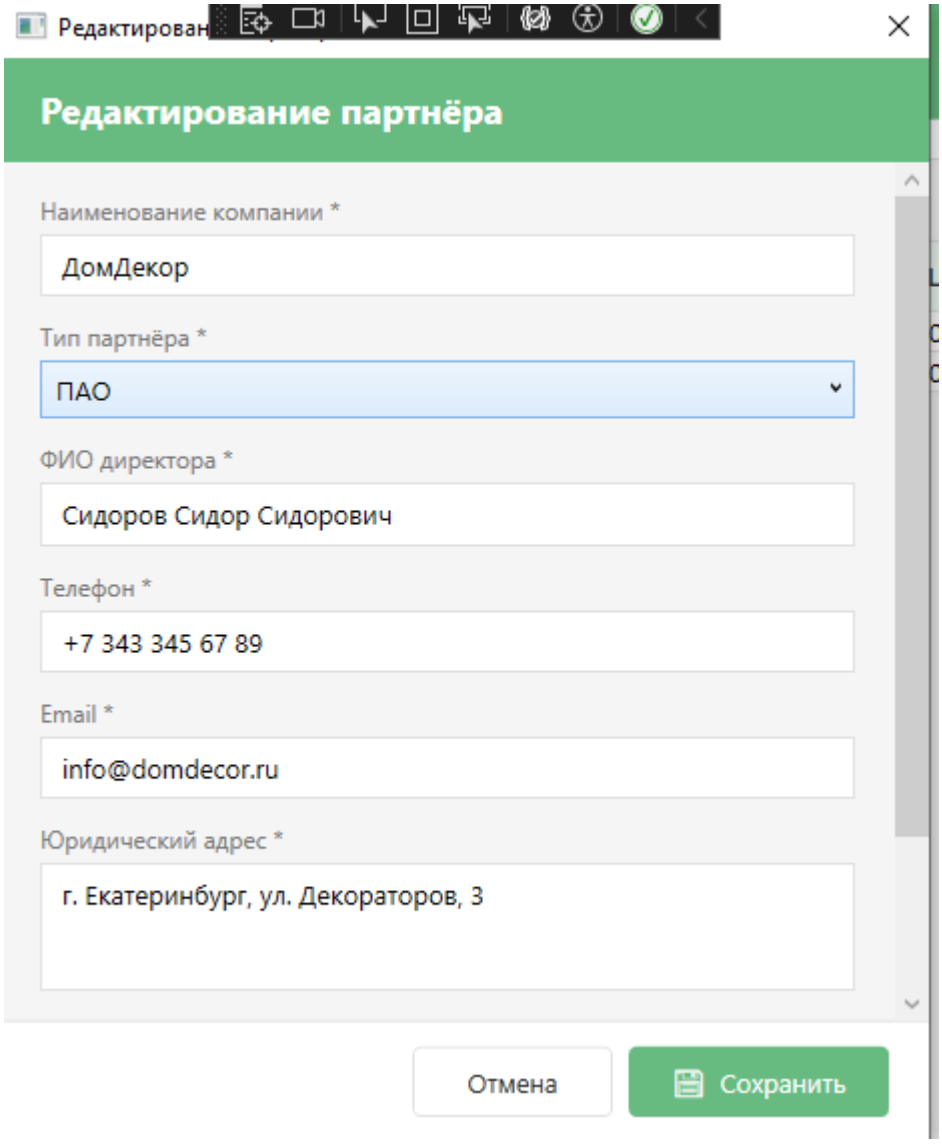

Рисунок 5 — Форма редактирования партнёра с заполненными полями

4.5. SalesWindow — окно управления продажами

Окно открывается через пункт меню «Справочники → Продажи». Отображает полную таблицу всех продаж всех партнёров с панелью инструментов.

Таблица продаж содержит столбцы: ID, партнёр, продукция, количество, дата, базовая цена, скидка%, цена продажи, итоговая сумма, комментарий.

После закрытия SalesWindow главное окно автоматически перезагружает список партнёров — скидки могут измениться из-за новых продаж.

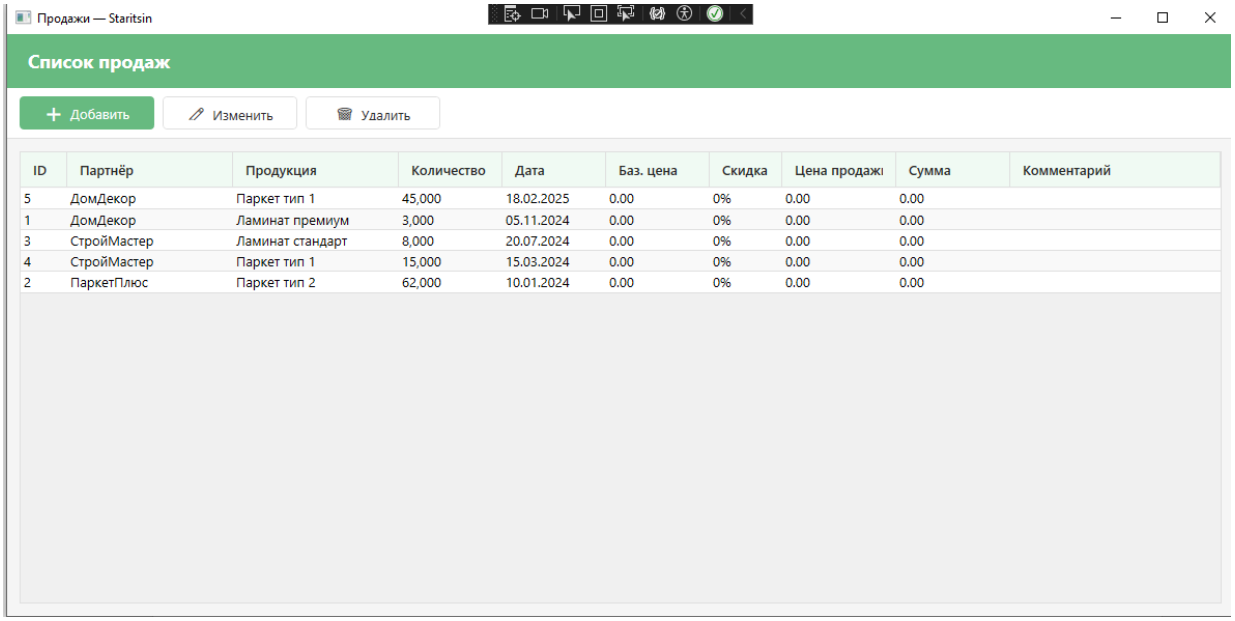

Рисунок 7 — Окно управления продажами SalesWindow с таблицей всех продаж

4.6. SaleEditWindow — форма продажи

Форма добавления и редактирования продажи. Ключевая особенность: при изменении любого поля (партнёр, продукт, количество) автоматически пересчитывается блок с ценами.

Автоматический расчёт в реальном времени

Обработчик изменения полей — автоматический пересчёт

```
private async void InputControl_Changed(object sender, EventArgs e)
{
    if (_isLoading) return;
    await RefreshCalculationAsync();
}

private async Task RefreshCalculationAsync()
{
    BasePriceText.Text = DiscountText.Text =
    UnitPriceText.Text = TotalAmountText.Text = "0.00";

    if (PartnerComboBox.SelectedValue == null ||
        ProductComboBox.SelectedValue == null) return;

    int quantity;
    if (!int.TryParse(QuantityBox.Text, out quantity)) return;
    if (quantity <= 0) return;
    var result = await _saleService.CalculateSaleAsync(
        (int)PartnerComboBox.SelectedValue,
        (int)ProductComboBox.SelectedValue,
        quantity, _editId);

    BasePriceText.Text   = result.BasePrice.ToString("N2");
    DiscountText.Text    = result.DiscountPercent + "%";
    UnitPriceText.Text   = result.UnitPrice.ToString("N2");
    TotalAmountText.Text = result.TotalAmount.ToString("N2");
}
```

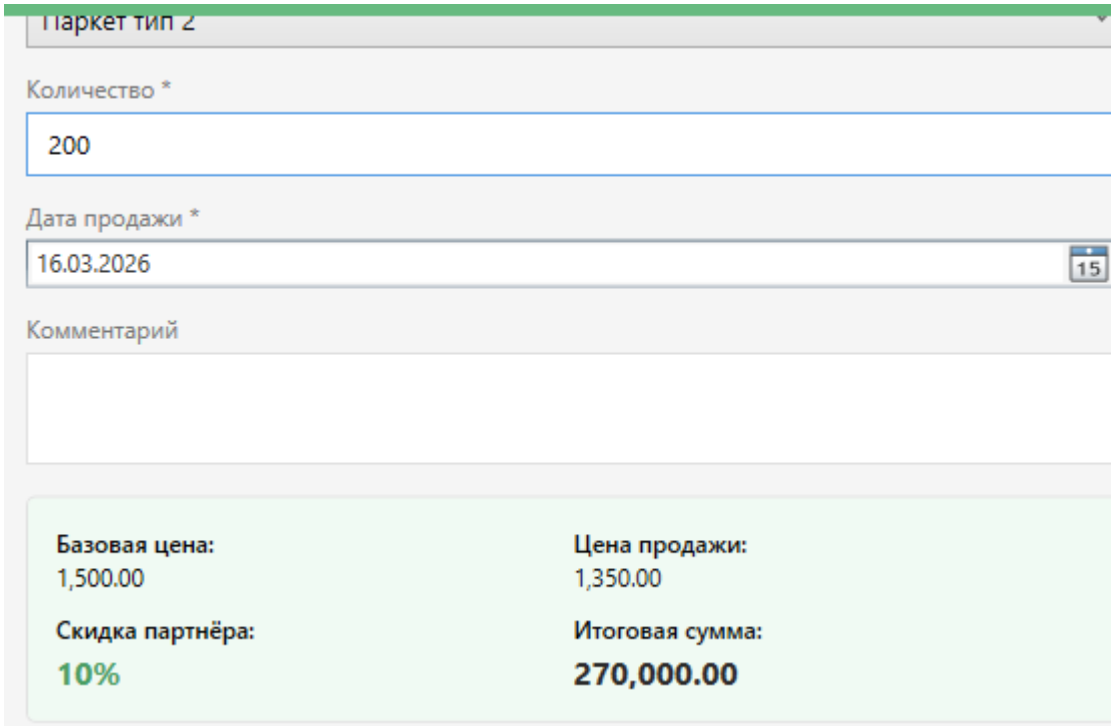

Рисунок 8 — Форма добавления продажи с автоматически рассчитанными ценой и суммой

5. Модульные тесты

Проект StaritsinPartners.Tests содержит 72 модульных теста, написанных с использованием фреймворка MSTest. Все тесты выполняются без базы данных и без запуска UI.

5.1. Структура тестового проекта

Тестовый класс	Тестов	Что проверяется
DiscountServiceTests	10	Все граничные значения алгоритма скидки: 0, 9999, 10000, 49999, 50000, 299999, 300000, 1000000
PartnerModelTests	10	Создание объекта, значения по умолчанию, присвоение всех свойств, навигационные свойства
PartnerTypeModelTests	4	Значения по умолчанию и присвоение свойств PartnerType
PartnerSaleModelUpdatedTests	10	Новые поля: BasePrice, DiscountPercent, UnitPrice, TotalAmount, Comment. Согласованность полей
ProductModelUpdatedTests	4	Новое поле Price в модели Product
SaleCalculationTests	13	Расчёт UnitPrice и TotalAmount при всех

		уровнях скидки, округление, полный цикл
SaleEditModelTests	7	Поля модели SaleEditModel, nullable Comment
SaleListItemDTOTests	7	Поля DTO списка продаж, TotalAmount = UnitPrice × Quantity
SaleCalculationResultDTOTests	7	Поля DTO результата, UnitPrice < BasePrice при скидке > 0

5.2. Паттерн Arrange-Act-Assert

Каждый тест следует паттерну AAA: Arrange (подготовка данных), Act (вызов тестируемого метода), Assert (проверка результата):

Пример теста — граничное значение 300 000 (15%)

```
[TestClass]
public class DiscountServiceTests
{
    private DiscountService _service;

    [TestInitialize]
    public void Setup()
    {
        _service = new DiscountService();
    }

    [TestMethod]
    public void CalculateDiscount_WhenQuantityIs300000_ReturnsFifteenPercent()
    {
```

```
        // Arrange — нижняя граница четвёртого диапазона
        int quantity = 300000;

        // Act
        int discount = _service.CalculateDiscount(quantity);

        // Assert
        Assert.AreEqual(15, discount);
    }
}
```

5.3. Тесты расчёта цены продажи

Класс SaleCalculationTests проверяет формулу UnitPrice = BasePrice × (1 − скидка/100) при всех уровнях скидки и граничных значениях:

Тест расчёта UnitPrice с учётом скидки 10%

```
[TestMethod]
public void UnitPrice_WhenDiscountIsTenPercent_CorrectlyReduced()
{
    // Arrange
    decimal basePrice       = 1000m;
    int      discountPercent = _discountService.CalculateDiscount(50000); //
10%

    // Act — имитируем формулу из SaleService
    decimal unitPrice = Math.Round(
        basePrice * (1m - discountPercent / 100m), 2,
        MidpointRounding.AwayFromZero);

    // Assert — 1000 * 0.90 = 900
    Assert.AreEqual(900.00m, unitPrice);
```

}

Количество (SUM)	Скидка	BasePrice	UnitPrice (ожидается)	TotalAmount (qty=5)
0	0%	1000.00	1000.00	5000.00
10 000	5%	1000.00	950.00	4750.00
50 000	10%	1000.00	900.00	4500.00
300 000	15%	1000.00	850.00	4250.00
62 000	10%	1200.00	1080.00	3240.00

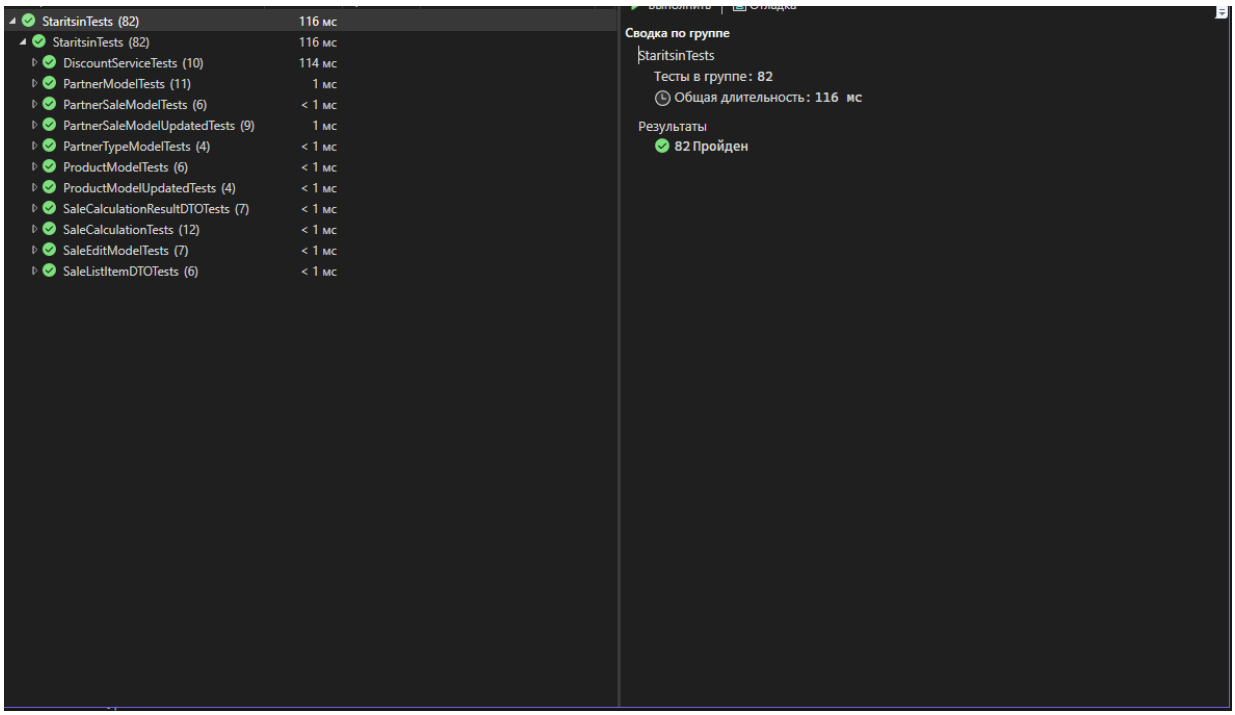

Рисунок 9 — Результаты запуска всех 72 тестов в Test Explorer Visual Studio (все зелёные)

Заключение

В ходе работы над проектом «StaritsinPartners» был разработан полноценный программный продукт, реализующий все требования технического задания.

Итоги разработки

Аспект	Результат
Архитектура	Трёхслойная архитектура с чётким разделением Domain/WPF/Tests. WPF-проект не содержит зависимостей от EF Core
База данных	PostgreSQL 15, схема app, 4 таблицы в 3НФ. Автоматическое создание при первом запуске
Бизнес-логика	Корректный алгоритм расчёта скидки 0/5/10/15% с сохранением исторических цен в продажах
Интерфейс	Единый визуальный стиль через App.xaml. 4 окна с валидацией, контекстным меню, поиском
Тестирование	72 модульных теста покрывают все граничные значения и основные сценарии. Все тесты проходят
Документация	XML-комментарии во всех публичных методах, README на GitHub, сопроводительная записка

Схема итогового решения

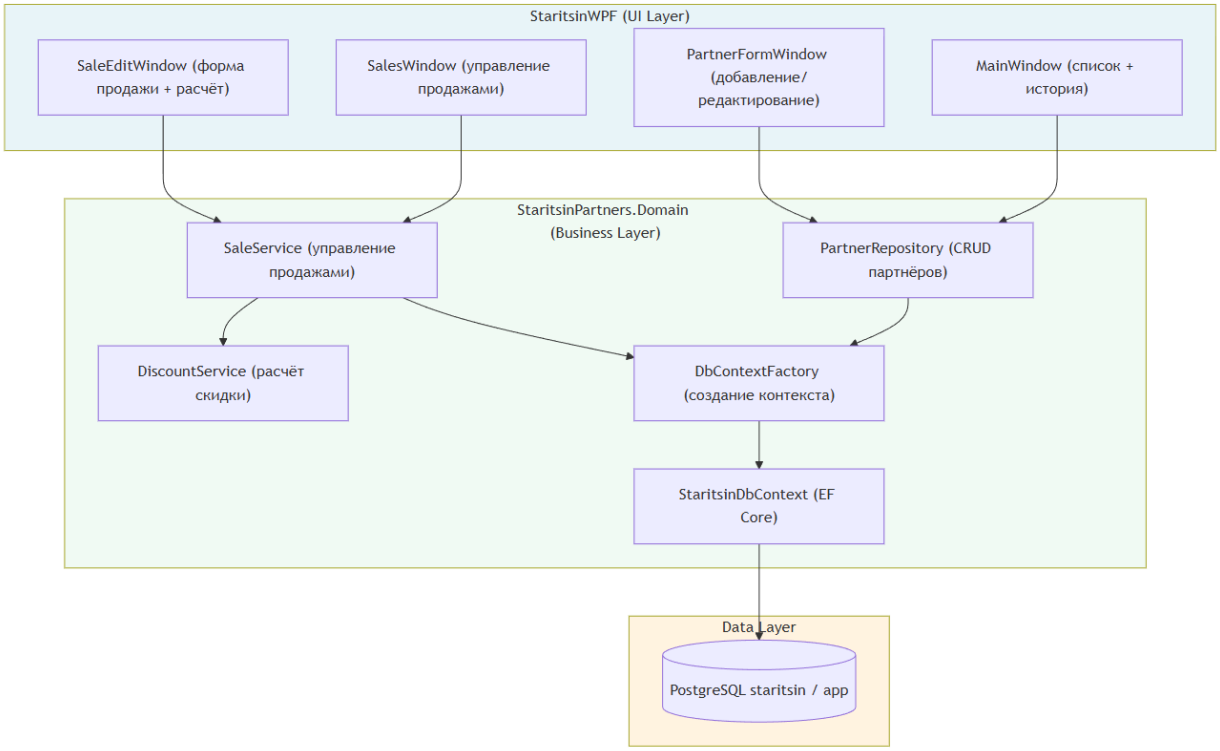

Диаграмма 11 — Итоговая схема взаимодействия компонентов

Разработанная подсистема корректно выполняет все заявленные функции: ведёт учёт партнёров, отображает историю реализации продукции, рассчитывает индивидуальные скидки и управляет продажами. Приложение устойчиво к ошибкам ввода — реализована валидация всех полей форм с информативными сообщениями.